Encontro 9: Integração com APIs e Consumo de Dados

Sumário

Encontro 9: Integração com APIs e Consumo de Dados 1
1. Introdução 2
2. O que é uma API RESTful? 2
   1.1. Front-end 2
   1.2. Endpoint 3
   1.3. Back-end 3
3. Onde Fazer Chamadas de API em Vue? 4
4. Método 1: Usando a fetch API 4
5. Método 2: Usando a Biblioteca axios 5
6. Fetch vs. Axios: Qual Usar? 5
7. Gerenciando Estados de UI (Carregando e Erro) 6
8. Exercício Prático Integrado: Buscador de Perfil do GitHub 7
   8.1. Solução Completa 7

1. Introdução

Até agora, nossas aplicações Vue têm trabalhado com dados locais, definidos diretamente no data dos componentes. No entanto, a maioria das aplicações do mundo real precisa buscar, exibir e enviar dados para um servidor remoto. É aqui que entram as APIs (Application Programming Interfaces).

2. O que é uma API RESTful?

Uma API é uma ponte que permite que diferentes sistemas de software se comuniquem. Uma **API RESTful** é um estilo de arquitetura de API muito popular que usa os métodos padrão do protocolo HTTP para realizar operações em recursos.

- Recurso: Qualquer tipo de objeto, dado ou serviço que pode ser acessado (ex: um usuário, um produto, uma lista de posts).
- Operações (Verbos HTTP):
  - `GET`: Para buscar/ler um recurso.
  - `POST`: Para criar um novo recurso.
  - `PUT` / `PATCH`: Para atualizar um recurso existente.
  - `DELETE`: Para remover um recurso.

Neste encontro, focaremos no método `GET`, que é usado para consumir dados de uma API.

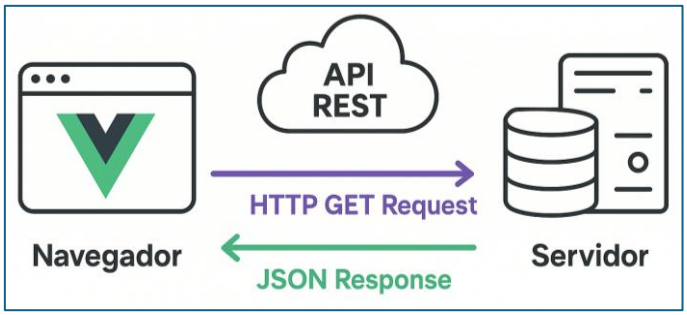


Figura 1: Fluxo de uma chamada a API REST

Um front-end Vue.js faz uma requisição HTTP para um endpoint da API, e o servidor responde com os dados, geralmente em formato JSON.

1.1. Front-end

O front-end refere-se à parte da aplicação com a qual o usuário interage diretamente. É a interface visual, tudo o que o usuário vê na tela do seu dispositivo (computador, smartphone, tablet, etc.) e com o qual pode interagir, como layouts, botões, menus, formulários e elementos gráficos.

- Foco principal: Experiência do Usuário (UX) e Interface do Usuário (UI).
- Tecnologias comuns: HTML (estrutura), CSS (estilo e layout) e JavaScript (interatividade e lógica do lado do cliente).
- Função: Transformar o design visual em uma interface funcional e responsiva, garantindo uma navegação intuitiva e agradável para o usuário.

1.2. Endpoint

Um endpoint é um ponto de comunicação específico em uma rede ou sistema de software. No desenvolvimento web e APIs, endpoint é a URL de um servidor ou serviço, para onde as solicitações do cliente (geralmente do front-end) são enviadas e onde o processamento ocorre.

- Foco principal: Localização e acesso a recursos ou funcionalidades no servidor (back-end).
- Tecnologias comuns: O back-end (onde os endpoints estão localizados) usa linguagens como Python, Java, PHP, Ruby e bancos de dados (MySQL, MongoDB, etc.).
- Função: Receber requisições do cliente, processar a lógica de negócios (por exemplo, verificar um login, consultar um banco de dados) e retornar uma resposta, geralmente em formato JSON.

1.3. Back-end

O back-end é a "parte de trás" ou o "motor" da aplicação, invisível para o usuário final. Lida com a lógica de negócios, o processamento de dados, a segurança e a comunicação com o banco de dados.

- O que faz: Recebe requisições do front-end, processa a lógica necessária, interage com o banco de dados para armazenar ou recuperar informações e envia as respostas de volta ao front-end.
- Tecnologias comuns: Linguagens de programação como Python, Java, PHP, Node.js, Ruby, e bancos de dados como MySQL, PostgreSQL, e MongoDB.
- Analogia: No restaurante, o back-end é a cozinha, onde os pedidos são processados, a comida é preparada e os ingredientes são gerenciados no estoque (banco de dados).

3. Onde Fazer Chamadas de API em Vue?

A prática mais comum é iniciar chamadas de API para buscar dados no hook de ciclo de vida `mounted`. Isso garante que a chamada seja feita assim que o componente for montado no DOM, permitindo que você exiba um estado de "carregando" enquanto os dados não chegam.

```
export default {
  data() {
    return {
      dados: null
    };
  },
  mounted() {
    // O melhor lugar para buscar dados iniciais
    this.buscarDadosDaAPI();
  },
  methods: {
    buscarDadosDaAPI() {
      // Lógica da chamada de API aqui
    }
  }
}
```

4. Método 1: Usando a fetch API

A `fetch API` é uma interface moderna, nativa dos navegadores, para fazer requisições de rede. Ela é baseada em Promises.

Exemplo com `async/await` (Recomendado)

`async/await` é uma sintaxe mais limpa e legível para trabalhar com Promises.

```
async buscarDadosComFetch() {
  try {
    const resposta = await fetch('https://api.exemplo.com/dados');
    if (!resposta.ok) {
      throw new Error("Erro na rede: " + resposta.statusText);
    }
    const dadosJson = await resposta.json(); // Converte a resposta para JSON
    this.dados = dadosJson;
  } catch (erro) {
    console.error("Houve um problema com a requisição fetch:", erro);
  }
}
```

5. Método 2: Usando a Biblioteca axios

`axios` é uma biblioteca de terceiros extremamente popular para fazer requisições HTTP. Ela simplifica muitas tarefas e oferece funcionalidades extras.

Primeiro, instale o `axios` no seu projeto:

```
npm install axios
```

Exemplo com `async/await`

```
import axios from 'axios';
async buscarDadosComAxios() {
  try {
    const resposta = await axios.get('https://api.exemplo.com/dados');
    this.dados = resposta.data; // Axios já converte a resposta para JSON
  } catch (erro) {
    console.error("Houve um problema com a requisição axios:", erro);
  }
}
```

6. Fetch vs. Axios: Qual Usar?

Ambos são excelentes, mas têm diferenças importantes.

Característica	`fetch API`	`axios`
Disponibilidade	Nativa do navegador (sem instalação)	Requer instalação (`npm install axios`)
Conversão de JSON	Manual (requer `resposta.json()`)	Automática
Tratamento de Erros	Não rejeita a Promise em erros HTTP (404, 500). Requer verificação manual (`resposta.ok`).	Rejeita a Promise em erros HTTP, caindo diretamente no bloco `catch`.
Compatibilidade	Navegadores modernos	Ampla compatibilidade (usa `XMLHttpRequest` por baixo dos panos)
Recursos Extras	Básica	Interceptadores, cancelamento de requisição, proteção XSRF, etc.

Recomendação: Para projetos mais simples, `fetch` é suficiente. Para projetos mais complexos, `axios` geralmente oferece uma experiência de desenvolvimento mais robusta e conveniente.

7. Gerenciando Estados de UI (Carregando e Erro)

Devemos sempre fornecer feedback visual ao usuário enquanto os dados estão sendo carregados ou se ocorrer um erro.

```
<template>
  <div>
    <div v-if="loading">Carregando...</div>
    <div v-else-if="error">{{ error }}</div>
    <div v-else>
      <!-- Exiba seus dados aqui -->
      <pre>{{ dados }}</pre>
    </div>
  </div>
</template>

<script>
import axios from 'axios';
export default {
  data() {
    return {
      loading: false,
      error: null,
      dados: null
    };
  },
  mounted() {
    this.buscarDados();
  },
  methods: {
    async buscarDados() {
      this.loading = true;
      this.error = null;
      try {
        const resposta = await axios.get('https://api.exemplo.com/dados');
        this.dados = resposta.data;
      } catch (err) {
        this.error = "Falha ao carregar os dados.";
      } finally {
        this.loading = false;
      }
    }
  }
}
</script>
```

8. Exercício Prático Integrado: Buscador de Perfil do GitHub

Objetivo: Criar uma aplicação que busca e exibe o perfil de um usuário do GitHub usando a API pública do GitHub.

Passo 1: Estrutura do Componente (`GithubProfileFinder.vue`)

Crie um componente com um campo de input para o nome de usuário e uma área para exibir os resultados.

Passo 2: Lógica da Requisição

Use `axios` para fazer uma requisição `GET` para `https://api.github.com/users/{username}`. Implemente os estados de `loading` e `error`.

8.1. Solução Completa

Componente `src/components/DogImageViewer.vue`

```
<template>
  <div class="profile-finder">
    <h2>Buscador de Perfil do GitHub</h2>
    <div class="search-bar">
      <input
        v-model="username"
        @keyup.enter="fetchUserProfile"
        type="text"
        placeholder="Digite um nome de usuário do GitHub"
      />
      <button @click="fetchUserProfile">Buscar</button>
    </div>

    <div v-if="loading" class="feedback">Carregando...</div>
    <div v-if="error" class="feedback error">{{ error }}</div>

    <div v-if="user" class="profile-card">
      <img :src="user.avatar_url" alt="Avatar do usuário" class="avatar" />
      <div class="user-info">
        <h3>{{ user.name }}</h3>
        <p class="login">@{{ user.login }}</p>
        <p>{{ user.bio }}</p>
        <a :href="user.html_url" target="_blank">Ver Perfil no GitHub</a>
      </div>
    </div>
  </div>
</template>

<script>
import axios from 'axios';
```

```
export default {
  data() {
    return {
      username: '',
      user: null,
      loading: false,
      error: null
    };
  },
  methods: {
    async fetchUserProfile() {
      if (!this.username) return;
      this.loading = true;
      this.error = null;
      this.user = null;
      try {
        const response = await
axios.get(`https://api.github.com/users/${this.username}`);
        this.user = response.data;
      } catch (err) {
        if (err.response && err.response.status === 404) {
          this.error = "Usuário não encontrado.";
        } else {
          this.error = "Ocorreu um erro ao buscar o perfil.";
        }
      } finally {
        this.loading = false;
      }
    }
  }
}
</script>

<style scoped>
.profile-finder {
  max-width: 500px;
  margin: 20px auto;
  padding: 20px;
  box-shadow: 0 2px 10px rgba(0,0,0,0.1);
  border-radius: 8px;
}
.search-bar { display: flex; margin-bottom: 20px; }
input {
  flex-grow: 1;
  padding: 10px;
  border: 1px solid #ccc;
  border-radius: 4px 0 0 4px;
}
button {
  padding: 10px 15px;
```

```
  border: none;
  background: #42b983;
  color: white;
  border-radius: 0 4px 4px 0;
  cursor: pointer;
}
.feedback { text-align: center; padding: 20px; }
.error { color: red; }
.profile-card {
  display: flex;
  align-items: center;
  background: #f9f9f9;
  padding: 20px;
  border-radius: 8px;
}
.avatar {
  width: 100px;
  height: 100px;
  border-radius: 50%;
  margin-right: 20px;
}
.user-info h3 { margin: 0; }
.user-info .login { color: #666; }
</style>
```

Arquivo `src/App.vue`

```
<template>
  <div id="app">
    <h1>Consumo de API com Vue.js</h1>
    <DogImageViewer />
  </div>
</template>

<script>
import DogImageViewer from './components/DogImageViewer.vue';
export default {
  components: {
    DogImageViewer
  }
}
</script>

<style>
#app {
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
  padding: 20px;
  background: #f5f5f5;
  min-height: 100vh;
}
```

```
h1 {
  text-align: center;
  color: #2c3e50;
}
</style>
```

Resultado Final



Figura 2: A aplicação final exibindo o perfil de um usuário do GitHub após a busca bem-sucedida.